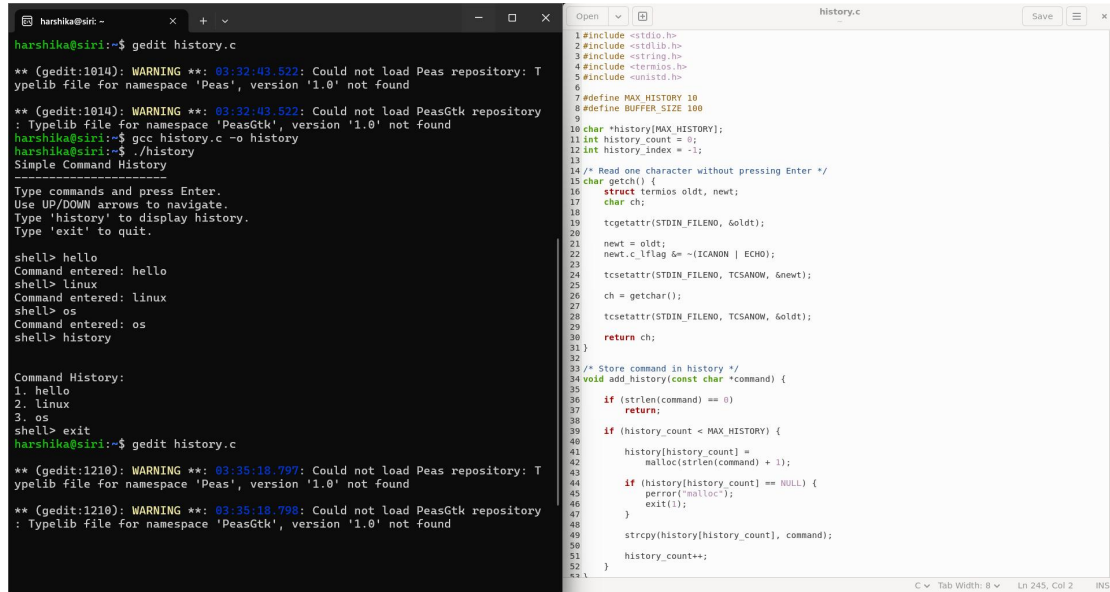


SKILL - 3

1.



The screenshot shows a terminal window on the left and a code editor on the right. The terminal window displays the execution of the `history.c` program, which demonstrates command history functionality. The code editor shows the source code of `history.c`, which includes standard headers, defines constants for history size and buffer size, and implements functions to get, add, and retrieve history entries.

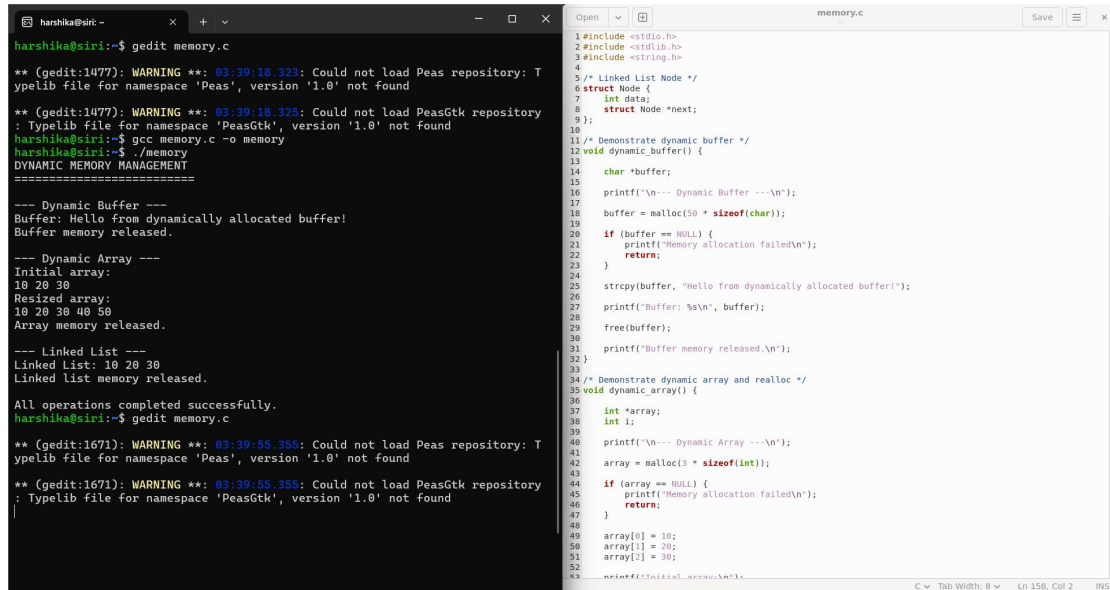
```
harshika@siri:~$ gedit history.c
** (gedit:1014): WARNING **: 03:32:43.522: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:1014): WARNING **: 03:32:43.522: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~$ gcc history.c -o history
harshika@siri:~$ ./history
Simple Command History
Type commands and press Enter.
Use UP/DOWN arrows to navigate.
Type 'history' to display history.
Type 'exit' to quit.

shell> hello
Command entered: hello
shell> linux
Command entered: linux
shell> os
Command entered: os
shell> history

Command History:
1. hello
2. linux
3. os
shell> exit
harshika@siri:~$ gedit history.c
** (gedit:1210): WARNING **: 03:35:18.797: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:1210): WARNING **: 03:35:18.798: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
```

```
1#include <stdio.h>
2#include <stdlib.h>
3#include <string.h>
4#include <termios.h>
5#include <unistd.h>
6
7#define MAX_HISTORY 10
8#define BUFFER_SIZE 100
9
10char *history[MAX_HISTORY];
11int history_count = 0;
12int history_index = -1;
13
14/* Read one character without pressing Enter */
15char getch() {
16    struct termios oldt, newt;
17    char ch;
18
19    tcgetattr(STDIN_FILENO, &oldt);
20
21    newt = oldt;
22    newt.c.lflag &= ~(ICANON | ECHO);
23
24    tcsetattr(STDIN_FILENO, TCSANOW, &newt);
25
26    ch = getchar();
27
28    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
29
30    return ch;
31}
32
33/* Store command in history */
34void add_history(const char *command) {
35
36    if (strlen(command) == 0)
37        return;
38
39    if (history_count < MAX_HISTORY) {
40
41        history[history_count] =
42            malloc(strlen(command) + 1);
43
44        if (history[history_count] == NULL) {
45            perror("malloc");
46            exit(1);
47        }
48
49        strcpy(history[history_count], command);
50
51        history_count++;
52    }
53}
```

2.



The screenshot shows a terminal window on the left and a code editor on the right. The terminal window displays the execution of the `memory.c` program, which demonstrates dynamic memory management using `malloc`, `realloc`, and `free`. The code editor shows the source code of `memory.c`, which includes standard headers, defines constants for buffer and array sizes, and implements functions to dynamically manage memory.

```
harshika@siri:~$ gedit memory.c
** (gedit:1477): WARNING **: 03:39:18.323: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:1477): WARNING **: 03:39:18.325: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~$ gcc memory.c -o memory
harshika@siri:~$ ./memory
DYNAMIC MEMORY MANAGEMENT
=====
--- Dynamic Buffer ---
Buffer: Hello from dynamically allocated buffer!
Buffer memory released.
--- Dynamic Array ---
Initial array:
10 20 30
Resized array:
10 20 30 40 50
Array memory released.
--- Linked List ---
Linked List: 10 20 30
Linked list memory released.
All operations completed successfully.
harshika@siri:~$ gedit memory.c
** (gedit:1671): WARNING **: 03:39:55.355: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:1671): WARNING **: 03:39:55.355: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
```

```
1#include <stdio.h>
2#include <stdlib.h>
3#include <string.h>
4
5/* Linked List Node */
6struct Node {
7    int data;
8    struct Node *next;
9};
10
11/* Demonstrate dynamic buffer */
12void dynamic_buffer() {
13
14    char *buffer;
15
16    printf("\n--- Dynamic Buffer ---\n");
17
18    buffer = malloc(50 * sizeof(char));
19
20    if (buffer == NULL) {
21        printf("Memory allocation failed\n");
22        return;
23    }
24
25    strcpy(buffer, "Hello from dynamically allocated buffer!");
26
27    printf("Buffer: %s\n", buffer);
28
29    free(buffer);
30
31    printf("Buffer memory released.\n");
32}
33
34/* Demonstrate dynamic array and realloc */
35void dynamic_array() {
36
37    int *array;
38    int i;
39
40    printf("\n--- Dynamic Array ---\n");
41
42    array = malloc(3 * sizeof(int));
43
44    if (array == NULL) {
45        printf("Memory allocation failed\n");
46        return;
47    }
48
49    array[0] = 10;
50    array[1] = 20;
51    array[2] = 30;
52
53    printf("Initial array:\n");
```